

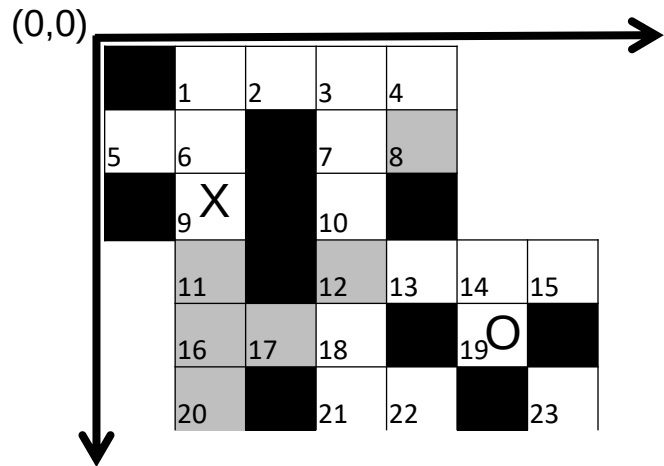
Objectiu de la classe:

Aprofundir el coneixement de com s'apliquen els algorismes de cerca informada en un problema de navegació amb obstacles.

Problema 1: Supposeu que heu de fer un agent que troba el camí que ha de seguir un robot per anar del punt on es troba (X:casella 9) al punt objectiu (O:casella 19), en un entorn com el del dibuix, les restriccions són:

- El robot no pot passar per les caselles negres, són caselles ocupades.
- El robot només pot fer passes en horitzontal o en vertical, mai en diagonal.
- El robot rep imatges d'una càmera zenital que permet extreure la distància des de la posició del robot fins a l'objectiu (*us donem una taula amb aquestes distàncies*)
- El robot consumeix energia en funció del color de la casella, amb les següents relacions:
 - (a) consum = $0.1 * \text{distància_recorreguda}$ (si les caselles són blanques)
 - (b) consum = $0.5 * \text{distància_recorreguda}$ (si les caselles són grises)
- El robot consumeix l'energia en el moment que surt de la casella.

Entorn del robot:



Taula de distàncies a l'objectiu:

Pos	dx	dy	d(Pos,19)
1	4	4	5,7
2	3	4	5,0
3	2	4	4,5
4	1	4	4,1
5	5	3	5,8
6	4	3	5,0
7	2	3	3,6
8	1	3	3,2
9	4	2	4,5
10	2	2	2,8
11	4	1	4,1
12	2	1	2,2

Pos	dx	dy	d(Pos,Obj)
13	1	1	1,4
14	0	1	1,0
15	1	1	1,4
16	4	0	4,0
17	3	0	3,0
18	2	0	2,0
19	0	0	0,0
20	4	1	4,1
21	2	1	2,2
22	1	1	1,4
23	1	1	1,4

Pos: Posició del Robot (Exemple: Pos(9)=(2,3))

Obj: Posició de l'Objectiu Obj=Pos(19)=(6,5)

dx: Distància horitzontal entre Pos i Obj

dy: Distància vertical entre Pos i Obj

d(Pos,Obj): Distància Euclidiana entre Pos i Obj

(Exemple: $d(\text{Pos}=10, \text{Obj}=19) = d((4,3), (6,5)) = \sqrt{dx^2 + dy^2} = \sqrt{2^2 + 2^2} = 2,8$)

Es demana fer un agent que permeti que el robot faci la trajectòria que impliqui el **menor consum d'energia possible**, per això contesteu a les següents preguntes:

1. Com seria l'arbre que representaria el problema, nodes?, branques?
2. Quins algorismes es poden aplicar?, i quin aplicaries?
3. Quin és la funció de cost d'aquest problema?
4. Es pot definir alguna heurística? Quina o quines?
5. Dibuixeu l'arbre que recorrerà l'A* des l'inici (Pos=9) fins que arribi al quadre 12 fent servir la millor heurística. Escriviu la seqüència de llistes.
6. Suposem que el Robot es pot moure en diagonal, posa un exemple d'un node en que l'h2 (basada en $d_{\text{Manhattan}}$) sobre-estima el cost de l'energia que cal fins a la solució
7. Què passaria si fas servir el GBFS en lloc de l'A*?, compareu el resultat obtingut amb els dos algorismes (considerant només moviments horitzontals i verticals).

Es demana fer un agent que permeti que el robot faci la trajectòria que impliqui el **menor consum d'energia possible**, per això contesteu a les següents preguntes:

1. Com seria l'arbre que representaria el problema, nodes?, branques?

Nodes: representen la posició del robot dins del mapa

Branques: representen el moviment del robot d'una casella a una altra

2. Quins algorismes es poden aplicar?, i quin aplicaries?

És un problema de cerca òptima.

La optimalitat ve definida per l'energia consumida pel robot no per la distància recorreguda.

Podem aplicar CCU o A*, els dos donen la solució òptima.

Aplicarem A* perquè gràcies a la càmera zenital es poden definir heurístiques i estimar la distància que queda fins a l'objectiu.

3. Quin és la funció de cost d'aquest problema?

Funció de cost:

$g(n)$ = Energia consumida des de l'arrel fins a n .

Es calcula sumant 0.1 o 0.5 per cada branca del camí, el color de la casella determina quina quantitat sumem.

4. Es pot definir alguna heurística? Quina o quines?

Podem definir dues possibles heurístiques admissibles:

$$\mathbf{h1(n)} = 0,1 * d_{\text{Euclidiana}}(n, \text{objectiu})$$

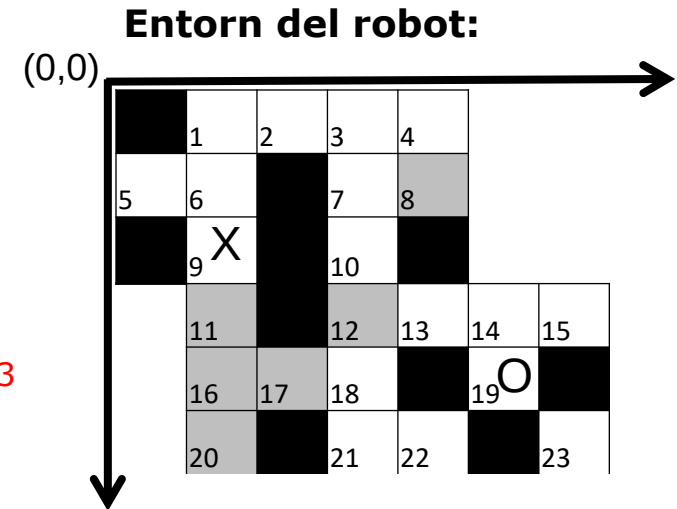
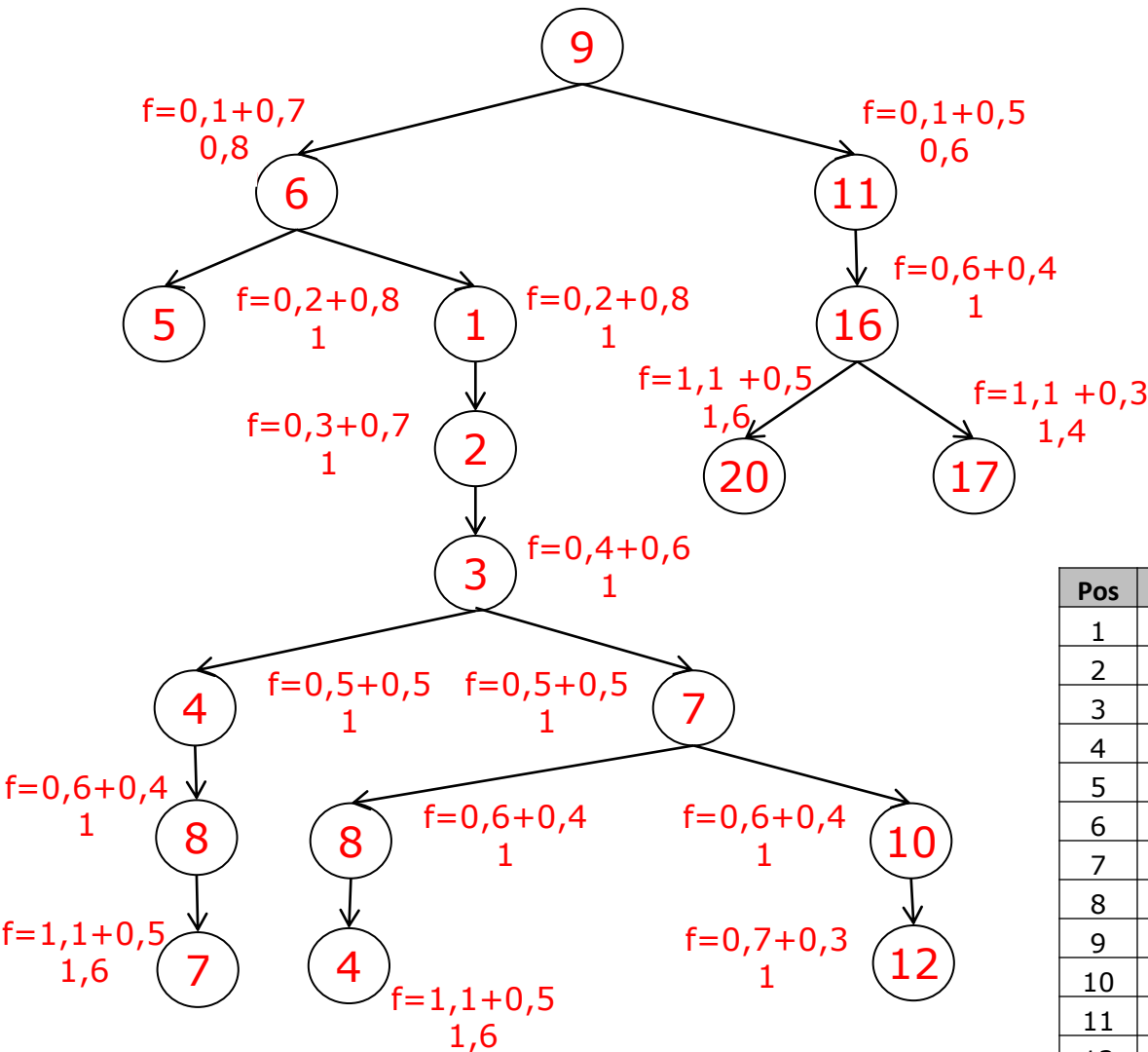
$$\mathbf{h2(n)} = 0,1 * d_{\text{Manhattan}}(n, \text{objectiu})$$

Criteris usat per definir-les:

Posem 0,1 per garantir que l'energia que estimem no estigui per sobre de l'energia real. Si posem 0,5 per l'estimació podríem sobre-estimar l'energia real òptima.

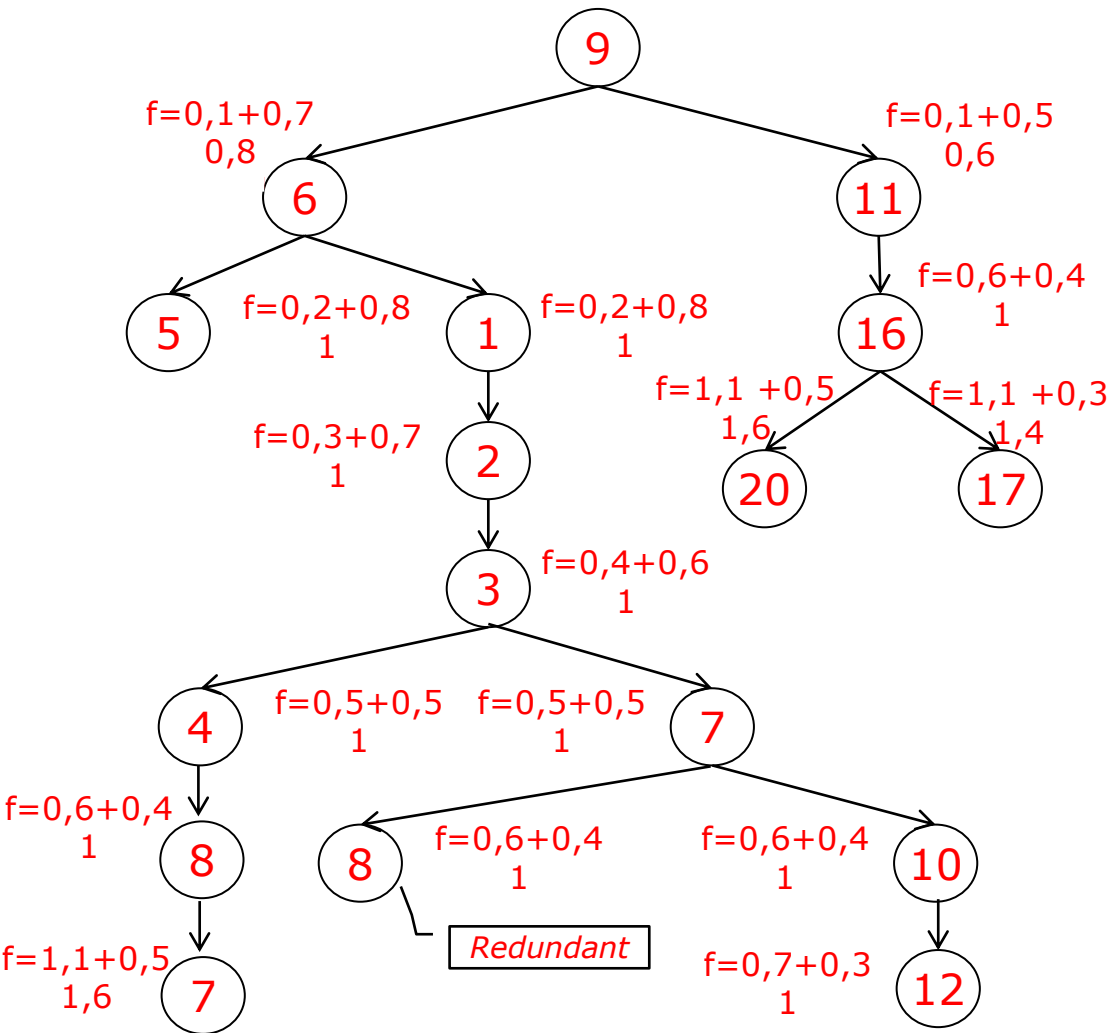
La $d_{\text{Manhattan}}$ és una estimació més gran que la $d_{\text{Euclidiana}}$ però com que el robot es mou només amb passes horitzontals i verticals (no en diagonal), aquesta distància permet calcular una sota-estimació dominant de l'energia real.

5. Dibuixeu l'arbre que recorrerà l'A* des l'inici (Pos=9) fins que arribi al quadre 12 fent servir la millor heurística. Escriviu la seqüència de llistes.



Pos	dx	dy	d(Pos,19)	Pos	dx	dy	d(Pos,Obj)
1	4	4	5,7	13	1	1	1,4
2	3	4	5,0	14	0	1	1,0
3	2	4	4,5	15	1	1	1,4
4	1	4	4,1	16	4	0	4,0
5	5	3	5,8	17	3	0	3,0
6	4	3	5,0	18	2	0	2,0
7	2	3	3,6	19	0	0	0,0
8	1	3	3,2	20	4	1	4,1
9	4	2	4,5	21	2	1	2,2
10	2	2	2,8	22	1	1	1,4
11	4	1	4,1	23	1	1	1,4
12	2	1	2,2				

La seqüència de llistes de l'arbre és la següent (fins a node 12):



Iter.0: $[[[9], 0.6]]$

Iter.1: $[[[11, 9], 0.6], [[6, 9], 0.8]]$

Iter.2: $[[[6, 9], 0.8], [[16, 11, 9], 1]]$

Iter.3: $[[[16, 11, 9], 1], [[5, 6, 9], 1],$
 $[[1, 6, 9], 1]]$

Iter.4: $[[[5, 6, 9], 1], [[1, 6, 9], 1],$
 $[[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6]]$

Iter.5: $[[[1, 6, 9], 1], [[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6]]$

Iter.6: $[[[2, 1, 6, 9], 1], [[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6]]$

Iter.7: $[[[3, 2, 1, 6, 9], 1], [[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6]]$

Iter.8: $[[[4, 3, 2, 1, 6, 9], 1], [[7, 3, 2, 1, 6, 9], 1],$
 $[[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6]]$

Iter.9: $[[[7, 3, 2, 1, 6, 9], 1], [[8, 4, 3, 2, 1, 6, 9], 1],$
 $[[17, 16, 11, 9], 1.4], [[20, 16, 11, 9], 1.6]]$

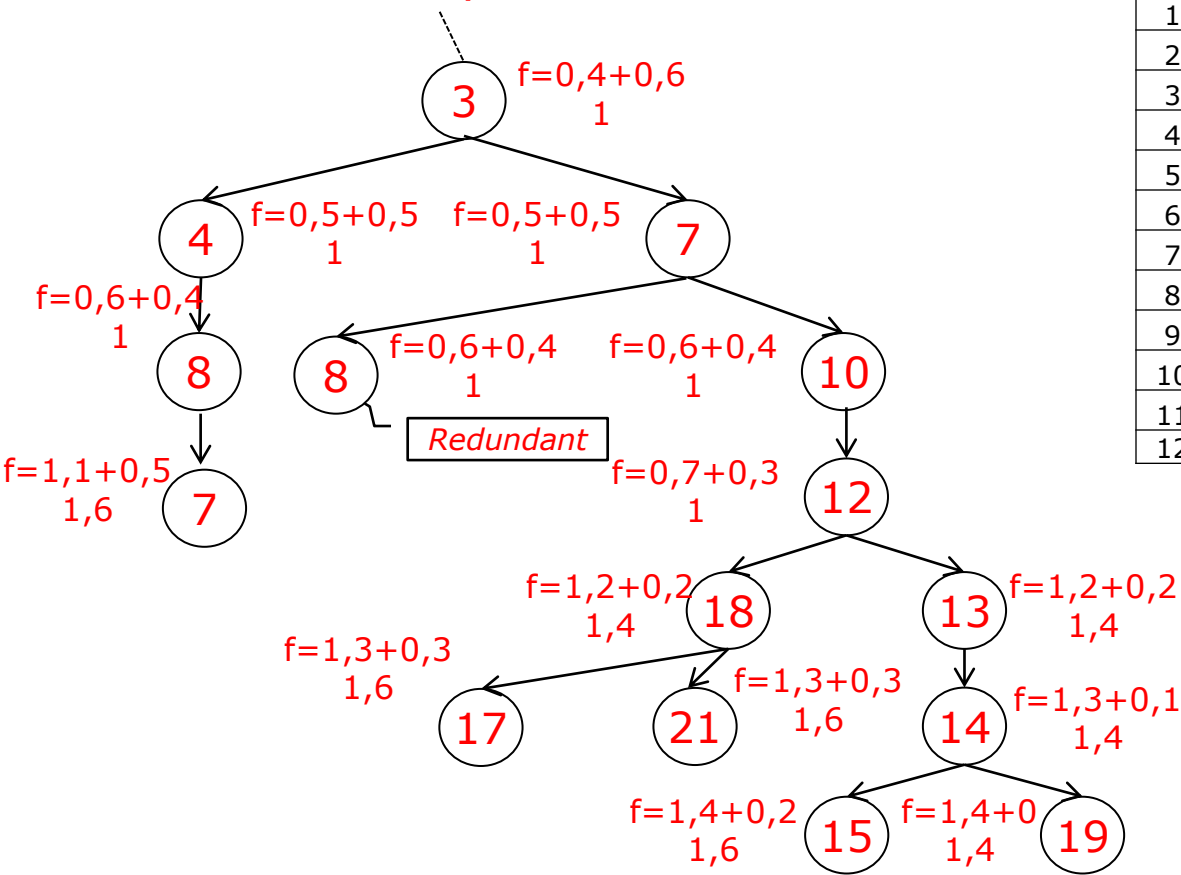
Iter.10: $[[[8, 4, 3, 2, 1, 6, 9], 1], [[8, 7, 3, 2, 1, 6, 9], 1],$
 $[[10, 7, 3, 2, 1, 6, 9], 1], [[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6]]$

Iter.11: $[[[10, 7, 3, 2, 1, 6, 9], 1], [[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6], [[7, 8, 4, 3, 2, 1, 6, 9], 1.6],$
 $[[4, 8, 7, 3, 2, 1, 6, 9], 1.6]]$

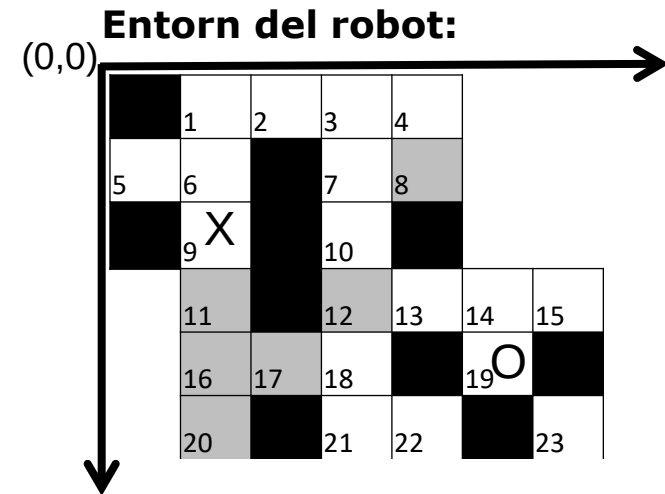
Iter.12: $[[[12, 10, 7, 3, 2, 1, 6, 9], 1], [[17, 16, 11, 9], 1.4],$
 $[[20, 16, 11, 9], 1.6], [[7, 8, 4, 3, 2, 1, 6, 9], 1.6],$
 $[[4, 8, 7, 3, 2, 1, 6, 9], 1.6]]$

Iter.13: return $[[12, 10, 7, 3, 2, 1, 6, 9], 1]$

... acabem l'A* que hem deixat al node 12

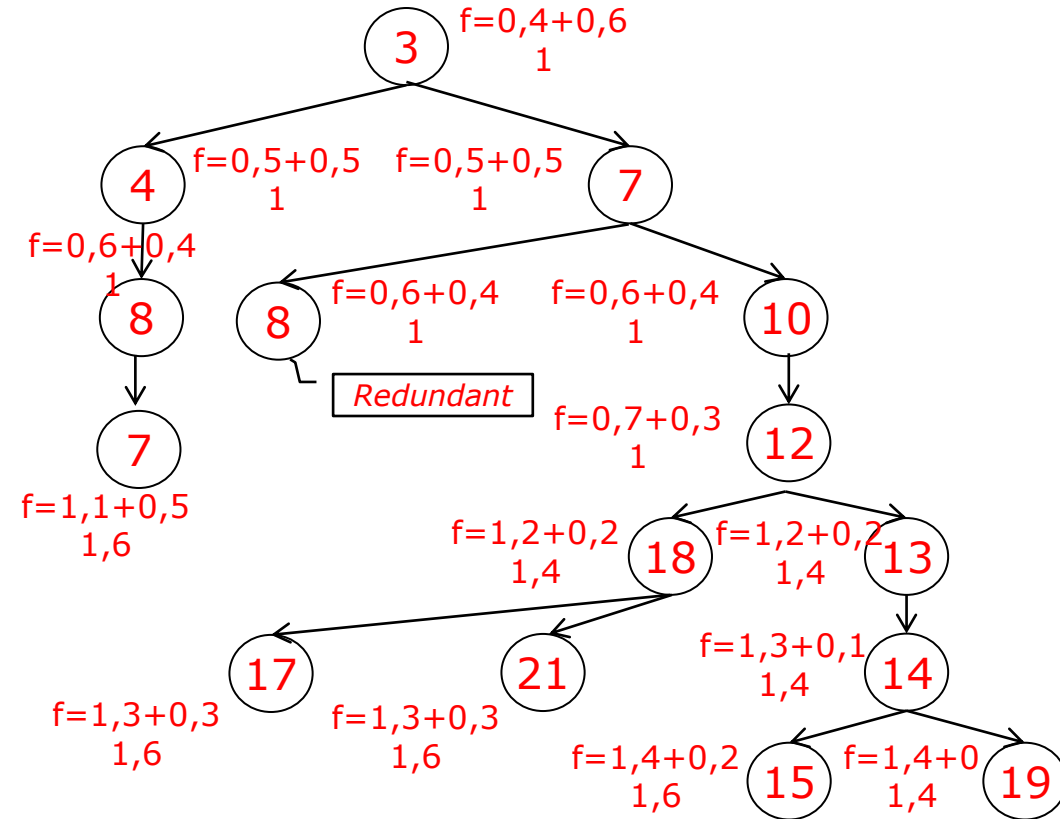


Pos	dx	dy	d(Pos,19)	Pos	dx	dy	d(Pos,Obj)
1	4	4	5,7	13	1	1	1,4
2	3	4	5,0	14	0	1	1,0
3	2	4	4,5	15	1	1	1,4
4	1	4	4,1	16	4	0	4,0
5	5	3	5,8	17	3	0	3,0
6	4	3	5,0	18	2	0	2,0
7	2	3	3,6	19	0	0	0,0
8	1	3	3,2	20	4	1	4,1
9	4	2	4,5	21	2	1	2,2
10	2	2	2,8	22	1	1	1,4
11	4	1	4,1	23	1	1	1,4
12	2	1	2,2				



Solució A*: [19,14,13,12,10, 7,3,2,1,6,9]

... La notació amb llistes



Iter.12: [[12,10,7,3,2,1,6,9],1.4],
 [[17,16,11,9],1.4],
 [[20,16,11,9],1.6],
 [[7,8,4,3,2,1,6,9],1.6],
 [[4,8,7,3,2,1,6,9],1.6]]

Iter.13: [[18,12,10,7,3,2,1,6,9],1.4],
 [[13, 12,10,7,3,2,1,6,9],1.4],
 [[17,16,11,9],1.4],
 [[20,16,11,9],1.6],
 [[7,8,4,3,2,1,6,9],1.6],
 [[4,8,7,3,2,1,6,9],1.6]]

Iter.14: [[13, 12,10,7,3,2,1,6,9],1.4],
 [[17,16,11,9],1.4],
 [[20,16,11,9],1.6],
 [[7,8,4,3,2,1,6,9],1.6],
 [[4,8,7,3,2,1,6,9],1.6],
 [[17,18,12,10,7,3,2,1,6,9],1.6],
 [[21,18,12,10,7,3,2,1,6,9],1.6]]

Iter.15: [[14, 13, 12,10,7,3,2,1,6,9],1.4],
 [[17,16,11,9],1.4],
 [[20,16,11,9],1.6],
 [[7,8,4,3,2,1,6,9],1.6],
 [[4,8,7,3,2,1,6,9],1.6],
 [[17,18,12,10,7,3,2,1,6,9],1.6],
 [[21,18,12,10,7,3,2,1,6,9],1.6]]

Iter.16: [[19, 14, 13, 12,10,7,3,2,1,6,9],1.4],
 [[17,16,11,9],1.4],
 [[20,16,11,9],1.6],
 [[7,8,4,3,2,1,6,9],1.6],
 [[4,8,7,3,2,1,6,9],1.6],
 [[17,18,12,10,7,3,2,1,6,9],1.6],
 [[21,18,12,10,7,3,2,1,6,9],1.6],
 [[15, 14, 13, 12,10,7,3,2,1,6,9],1.6]]

Iter.17: return [[19,14,13, 12,10,7,3,2,1,6,9],1.4]

6. Suposem que el Robot es pot moure en diagonal, posa un exemple d'un node en que l'h2 (basad en $d_{\text{Manhattan}}$) sobre-estima el cost de l'energia que cal fins a la solució:

$$h2(n) = 0,1 * d_{\text{Manhattan}}(n, \text{objectiu})$$

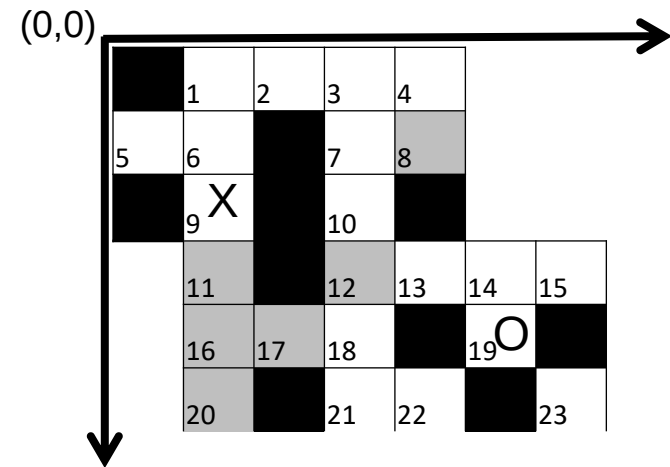
Exemple 1: Pos 13

El valor de h2 des la Pos=13 a la Pos=19 és

$$h2(13) = 0,1 * d_{\text{Manhattan}}(13, 19) = 0,1 * 2 = \mathbf{0,2}$$

El cost real òptim d'anar de Pos=13 a Pos=19 és un moviment en diagonal

$$\begin{aligned} g_{\text{òptim}}(13 \text{ a } 19) &= 0,1 * d_{\text{recoreguda}}(13, 19) = \\ &= 0,1 * d_{\text{Euclidiana}}(13, 19) = 0,1 * \sqrt{dx^2 + dy^2} = \\ &= 0,1 * \sqrt{1^2 + 1^2} = 0,1 * \sqrt{2} = 0,1 * 1,4 = \mathbf{0,14} \end{aligned}$$



$$\mathbf{0,2 > 0,14}$$

Exemple 2: Pos 10

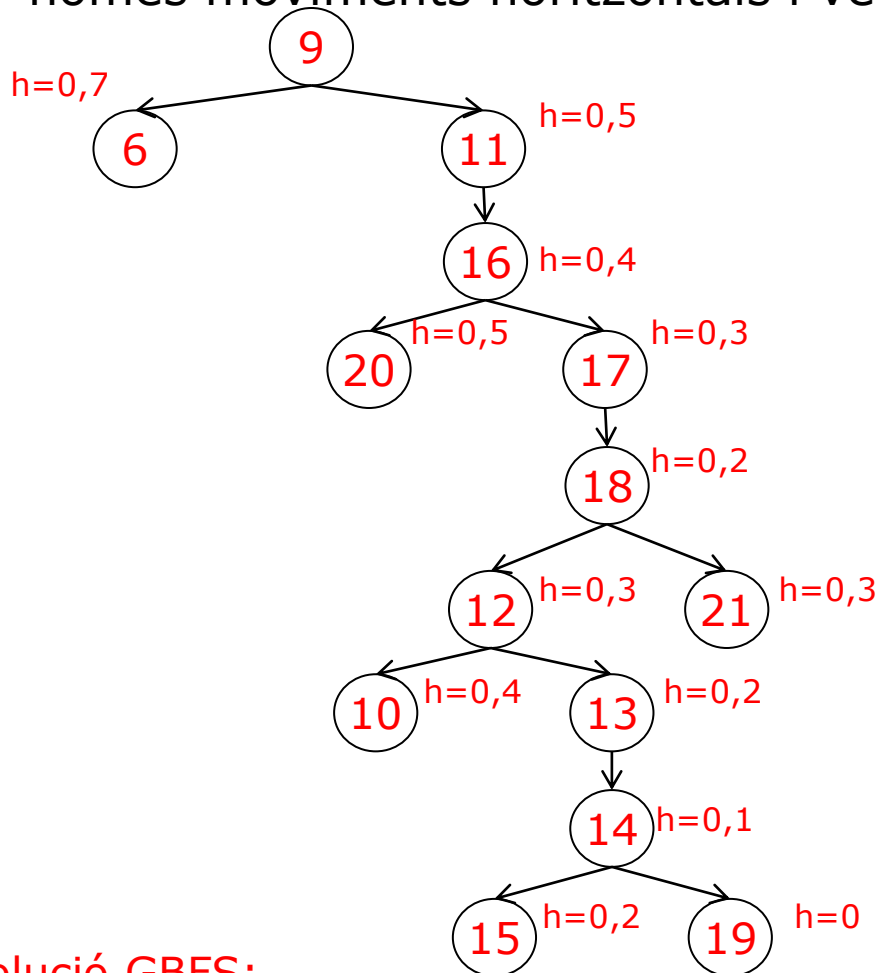
$$h2(10) = 0,1 * d_{\text{Manhattan}}(10, 19) = 0,1 * 4 = \mathbf{0,4}$$

$$\begin{aligned} g_{\text{òptim}}(10 \text{ a } 19) &= 0,1 * d_{\text{recoreguda}}(10, 19) = 0,1 * d_{\text{Euclidiana}}(10, 19) = 0,1 * \sqrt{dx^2 + dy^2} = \\ &= 0,1 * \sqrt{2^2 + 2^2} = 0,1 * \sqrt{8} = 0,1 * 2,8 = \mathbf{0,28} \end{aligned}$$

$$\mathbf{0,4 > 0,28}$$

7. Què passaria si fas servir el GBFS en lloc de l'A*?,

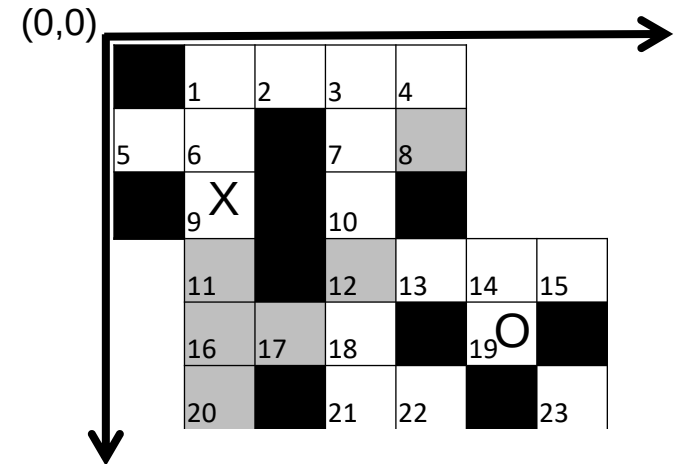
Compareu el resultat obtingut amb els dos algorismes (considerant només moviments horitzontals i verticals).



Solució GBFS:

[19,14,13,12,18,17,16,11,9]

Entorn del robot:



Pos	dx	dy	d(Pos,19)	Pos	dx	dy	d(Pos,Obj)
1	4	4	5,7	13	1	1	1,4
2	3	4	5,0	14	0	1	1,0
3	2	4	4,5	15	1	1	1,4
4	1	4	4,1	16	4	0	4,0
5	5	3	5,8	17	3	0	3,0
6	4	3	5,0	18	2	0	2,0
7	2	3	3,6	19	0	0	0,0
8	1	3	3,2	20	4	1	4,1
9	4	2	4,5	21	2	1	2,2
10	2	2	2,8	22	1	1	1,4
11	4	1	4,1	23	1	1	1,4
12	2	1	2,2				

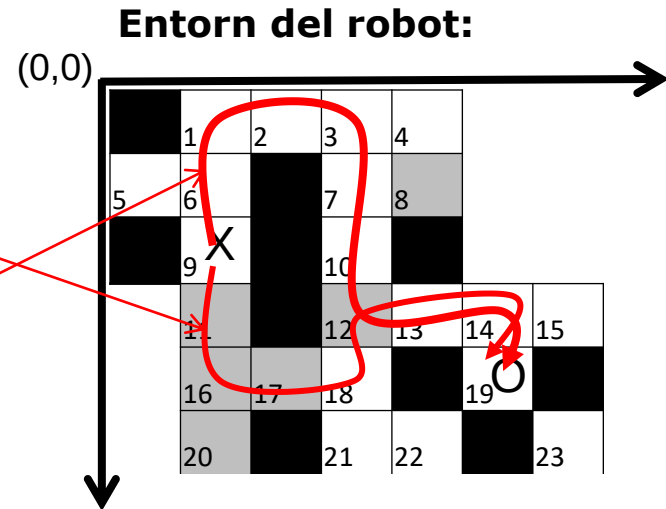
Comparem les dues solucions:

L'algorisme GBFS dóna com a solució:

[19,14,13,12,18,17,16,11,9]

L'algorisme A* dóna com a solució:

[19,14,13,12,10,7,3,2,1,6,9]



La solució que dóna el GBFS és més curta (9) però gasta més energia (2.5):

- 4 caselles grises -> Energia=0.5*4=2 Distància recorreguda=4
- 5 caselles blanques -> Energia=0.1*5=0.5 Distància recorreguda=5

La solució que dóna l'A* és més llarga (11) però gasta menys energia (1.5):

- 1 casella grisa -> Energia=0.5*1=0.5 Distància recorreguda=1
- 10 caselles blanques -> Energia=0.1*10=1 Distància recorreguda=10